

Отчет по лабораторной работе № 11 по курсу “Фундаментальная информатика”

Студент группы М80-107Б-20 Сысоев Максим Алексеевич, № по списку 23

Контакты e-mail, telegram, skype
masysoev@mai.education

Работа выполнена: «16 » ноября 2020г.

Преподаватель: каф. 806 Найденов Иван Евгеньевич

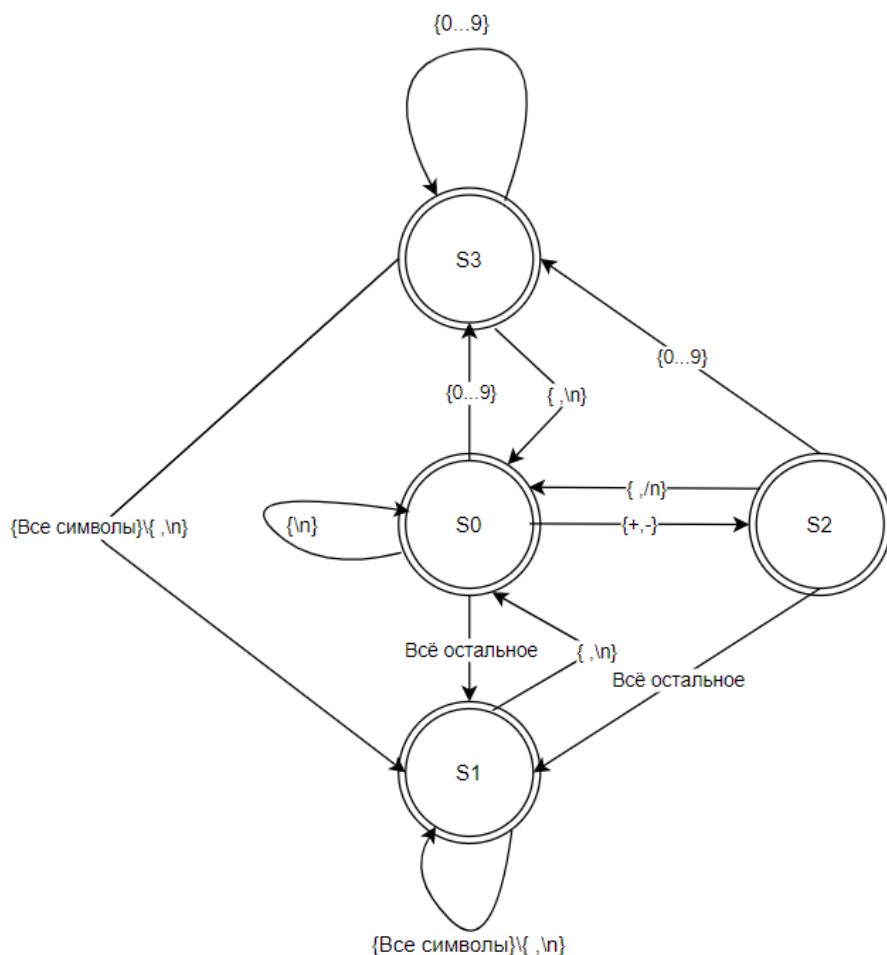
Отчет сдан <> ____ 20__ г., итоговая оценка ____

Подпись преподавателя

- 1 **Тема:** Обработка последовательности литер входного текстового файла. Простейшие приёмы лексического анализа. Диаграммы состояний и переходов.
- 2 **Цель работы:** Составить программу на Си, выполняющую анализ и обработку вводимого текста.
- 3 **Задание (вариант № 26):** Составить программу, выполняющую анализ и обработку вводимого аscii-текста. Требуется построчно выделить предпоследнее десятичное число и вычислить сумму его цифр. При решении задачи необходимо использовать матаппарат конечных автоматов. Запрещено использовать массивы, указатели и явные числовые коды аscii-знаков (например, число 48 вместо литерной константы '0').
- 4 **Оборудование (Студента):**
Процессор AMD Ryzen 3 3200U @ 2.60 GHz с ОП 8 Гб, НМД 256 Гб. Монитор 1920x1080
- 5 **Программное обеспечение (Студента):**
Операционная система семейства: linux, наименование: ubuntu версия 14.1.0 интерпретатор команд: bash версия 4.4.19.
Система программирования -- версия --
Редактор текстов: emacs
Утилиты операционной системы --
Прикладные системы и программы: VS 2019
Местонахождение и имена файлов программ и данных на домашнем компьютере--

6. **Идея, метод, алгоритм** решения задачи (в формах: словесной, псевдокода, графической [блок-схема, диаграмма, рисунок, таблица] или формальные спецификации с пред- и постусловиями)

Схема конечного автомата, на основе которого решена задача:



Регулярное выражение, по которому ищется нужное число:

[+,-]?[0-9]+

7. Сценарий выполнения работы [план работы, первоначальный текст программы в черновике (можно на отдельном листе) и тесты либо соображения по тестированию].

Тесты для программы:

Входные данные	Выходные данные	Описание тестирующего случая
15 16	6	Классический случай
+10 -19a	Пусто	В числе встречается мусор
+192 0 bfg	12	Обработка чисел со знаком
12f gf10 a = 10	Пусто	Всего одно число
10 12a ' ' 91	Пусто	Обработка случая, когда в конце пробел

Пункты 1-7 отчета составляются строго до начала лабораторной работы.

8. Распечатка протокола (подклеить листинг окончательного варианта программы с тестовыми примерами, подписанный преподавателем).

```
#include <stdio.h>

typedef enum {
    S0, S1, S2, S3
} State;

int is_minus_or_plus(char c)
{
    if (c == '-' || c == '+') {
        return 1;
    }
    return 0;
}

int is_rubbish(char c)
{
    if ((c < '0' && c >= '!') || (c <= '~' && c > '9')) {
        return 1;
    }
    return 0;
}

int is_number(char c)
{
    if (c <= '9' && c >= '0') {
        return 1;
    }
    return 0;
}

int is_space(char c)
{
    if (c == ' ') {
        return 1;
    }
    return 0;
}

int is_n(char c)
{
    if (c == '\n') {
        return 1;
    }
    return 0;
}
```

```

int main(void)
{
    char c;
    int sum1, sum2, sum3;
    sum1 = sum2 = sum3 = -1;
    State state = S0;

    while ((c = getchar()) != EOF) {
        switch (state) {
            case S0: // Начинается обработка нового слова
                if (is_minus_or_plus(c)) {
                    state = S2;
                } else if (is_rubbish(c)) {
                    state = S1;
                } else if (is_number(c)) {
                    state = S3;
                    sum3 = sum2;
                    sum2 = sum1;
                    sum1 = 0;
                    sum1 += c - '0';
                } else if (is_n(c)) {
                    if (sum1 != -1 && sum2 != -1) {
                        printf("%d\n", sum2);
                    }
                    sum1 = sum2 = sum3 = -1;
                    state = S0;
                }
                break;
            case S1: // Пропускает слово, ибо оно не число
                if (is_rubbish(c) || is_number(c)) {
                    state = S1;
                } else if (is_space(c)) {
                    state = S0;
                } else if (is_n(c)) {
                    if (sum1 != -1 && sum2 != -1) {
                        printf("%d\n", sum2);
                    }
                    sum1 = sum2 = sum3 = -1;
                    state = S0;
                }
                break;
            case S2: // Проверка, что + или - знаки числа, а не мусор
                if (is_number(c)) {
                    sum3 = sum2;
                    sum2 = sum1;
                    sum1 = 0;
                    sum1 += c - '0';
                    state = S3;
                } else if (is_space(c)) {

```

```

        state = S0;
    } else if (is_rubbish(c)) {
        state = S1;
    } else if (is_n(c)) {
        if (sum1 != -1 && sum2 != -1) {
            printf("%d\n", sum2);
        }
        sum1 = sum2 = sum3 = -1;
        state = S0;
    }
    break;
case S3: // Обработка числа
    if (is_number(c)) {
        sum1 += c - '0';
        state = S3;
    } else if (is_rubbish(c)) {
        sum1 = sum2;
        sum2 = sum3;
        state = S1;
    } else if (is_space(c)) {
        state = S0;
    } else if (is_n(c)) {
        if (sum1 != -1 && sum2 != -1) {
            printf("%d\n", sum2);
        }
        sum1 = sum2 = sum3 = -1;
        state = S0;
    }
    break;
}
}
return 0;
}

```

9. Дневник отладки должен содержать дату и время сеансов отладки и основные события (ошибки в сценарии и программе, нестандартные ситуации) и краткие комментарии к ним. В дневнике отладки приводятся сведения об использовании других ЭВМ, существенном участии преподавателя и других лиц в написании и отладке программы.

№	Лаб. или дом.	Дата	Время	Событие	Действие по исправлению	Примечание

10. Замечания автора по существу работы

11. Выводы

Классная лабораторная работа. Много нужно учитывать при написании программы, предусматривать все возможные переходы из разных состояний. Знание конечных автоматов и регулярных выражений, определённо может пригодиться в будущем, например в геймдэве(кон.автоматы) или для поиска(рег.выражения). Научился рисовать схемы конечных автоматов, составлять регулярные выражения. В целом, очень полезная и увлекательная работа.

Недочёты при выполнении задания могут быть устранены следующим образом:

Подпись студента
